

Task 1: Performance Analysis of Different Tokenization Tools

Informal English Social Media Posts

Experimental Study Report

Evaluation of NLTK TweetTokenizer, spaCy, and NLTK Word Tokenize

Abstract. This study evaluates three tokenization approaches on 50 unique English-language social-media posts selected from a public Tesla-related Twitter/X dataset. The experiment focuses on preservation of hashtags, mentions, URLs, and emojis, together with the total number of tokens generated. Ground truth was extracted from the original posts before tokenization using regular expressions and *emoji.emoji_list()*. NLTK TweetTokenizer achieved 100% preservation of hashtags, mentions, and URLs and 95.35% emoji preservation. spaCy achieved 97.78% URL preservation and 95.35% emoji preservation. NLTK Word Tokenize generated the most tokens, 2,151, but showed substantially weaker preservation of social-media-specific elements.

1. Introduction

Social-media text differs from conventional written English. Posts commonly contain hashtags, user mentions, URLs, emojis, abbreviations, slang, misspellings, and other informal constructions. These features create challenges for tokenization because a tokenizer must determine whether such elements should remain intact or be divided into multiple tokens. This experiment compares three tokenization approaches to determine their effectiveness on informal social-media text.

2. Objectives

- Determine which tokenizer performs best on hashtags.
- Determine which tokenizer preserves emojis most effectively.
- Determine which tokenizer generates the greatest number of tokens.
- Identify the strengths and weaknesses of each tokenizer for noisy social-media text.

3. Dataset

The source dataset contains 10,016 Twitter/X posts concerning Tesla. For the experiment, 50 unique English-language posts were selected from the source dataset. The original dataset was retained separately, while the experimental subset was saved as `data/task1_posts.csv`. The study focuses on the raw post text rather than relying on tokenizer-provided extracted fields.

Dataset note: Because the source corpus is Tesla-focused, the vocabulary and topics may not represent all forms of social-media communication. This is considered a limitation of the experiment.

4. Ground Truth Creation

Ground truth was created before any tokenizer was applied. This prevents the definition of correctness from being influenced by tokenizer output. Hashtags, mentions, and URLs were extracted with regular expressions, while emojis were identified using the *emoji* Python library.

Element	Extraction method	Example
Hashtag	(?<!w)#[A-Za-z0-9_]+	#Tesla
Mention	(?<!w)@[A-Za-z0-9_]+	@elonmusk
URL	https?:\/\/[^\s]+ www\.[^\s]+	https://example.com
Emoji	emoji.emoji_list(text)	■ / ■

5. Tokenization Tools

5.1 NLTK TweetTokenizer

TweetTokenizer is designed for tweet-style text. It was configured with `preserve_case=True`, `reduce_len=False`, and `strip_handles=False`. This configuration was chosen to retain social-media structures and avoid reducing repeated characters.

5.2 spaCy

The spaCy English model `en_core_web_sm` was used. spaCy provides general-purpose linguistic tokenization and was included as a comparison against the tweet-specific tokenizer.

5.3 NLTK Word Tokenize

NLTK's standard `word_tokenize()` function was evaluated as a conventional text tokenizer. It provides a useful baseline for comparison with the social-media-oriented TweetTokenizer.

6. Experimental Procedure

1. Load the public source dataset.
2. Filter to English-language posts and remove duplicate post text.
3. Select 50 experimental posts.
4. Extract ground-truth hashtags, mentions, URLs, and emojis from the original text.
5. Apply the same 50 posts to all three tokenizers.
6. Save detailed per-post tokenizer output to TXT and structured results to CSV.
7. Compare tokenizer output with ground truth.
8. Calculate preservation percentages and token counts.
9. Inspect failures to identify tokenizer-specific strengths and weaknesses.

7. Evaluation Definition

A social-media element was considered correctly preserved when the complete ground-truth string appeared as a single tokenizer token. For example, #Tesla was considered preserved when it appeared as ['#Tesla'], but not when it was split into ['#', 'Tesla']. The same principle was applied to mentions and URLs. For emojis, the complete emoji sequence returned by *emoji.emoji_list()* had to occur as one token.

8. Experimental Results

Tokenizer	Hashtag %	Mention %	URL %	Emoji %	Total Tokens	Avg. /Post
NLTK TweetTokenizer	100.00	100.00	100.00	95.35	1,814	36.28
spaCy	0.00	0.00	97.78	95.35	2,013	40.26
NLTK Word Tokenize	0.00	0.00	0.00	34.88	2,151	43.02

Note: These are the actual measurements produced by the experimental code for the selected 50 posts.

9. Analysis of Results

9.1 Hashtags

NLTK TweetTokenizer achieved 100% hashtag preservation, while spaCy and NLTK Word Tokenize achieved 0%. TweetTokenizer therefore performed best for the hashtag-specific requirement. The observed failure mode of the other tokenizers was splitting a hashtag such as #Tesla into separate symbol and word tokens.

9.2 Mentions

TweetTokenizer also achieved 100% mention preservation. spaCy and NLTK Word Tokenize achieved 0%. A representative failure is the separation of a handle such as @elonmusk into @ and elonmusk.

9.3 URLs

TweetTokenizer achieved 100% URL preservation, followed by spaCy at 97.78%. NLTK Word Tokenize achieved 0%. The conventional tokenizer can divide a URL into components such as the protocol, punctuation, and domain/path, which is undesirable when the URL itself is intended to be a single token.

9.4 Emojis

TweetTokenizer and spaCy tied at 95.35% emoji preservation. NLTK Word Tokenize achieved 34.88%. The remaining errors are consistent with the difficulty of handling complex Unicode emoji sequences, including sequences containing variation selectors or zero-width joiners.

9.5 Token Generation

NLTK Word Tokenize generated the most tokens: 2,151 in total, or 43.02 tokens per post. spaCy generated 2,013 tokens, while TweetTokenizer generated 1,814. A higher token count does not necessarily indicate better tokenization because splitting a single social-media unit into multiple pieces can increase the token count.

10. Strengths and Weaknesses

Tokenizer	Strengths	Weaknesses
NLTK TweetTokenizer	100% hashtag, mention, and URL preservation; 95.35% emoji preservation; designed for tweet-style text.	Some complex emoji sequences may still be split.
spaCy	97.78% URL preservation and 95.35% emoji preservation; strong general-purpose NLP capabilities.	Did not preserve hashtags and mentions as complete tokens in this experiment.
NLTK Word Tokenize	Produces fine-grained tokenization and the highest token count.	Poor preservation of hashtags, mentions, URLs, and emojis; can over-split social-media structures.

11. Answers to the Assignment Questions

1. Which tokenizer performs best on hashtags?

NLTK TweetTokenizer, with 100.00% preservation.

2. Which tokenizer preserves emojis correctly?

NLTK TweetTokenizer and spaCy tied for the highest measured emoji preservation at 95.35%.

3. Which tokenizer generates the most tokens?

NLTK Word Tokenize, with 2,151 total tokens and 43.02 tokens per post.

4. What are the strengths and weaknesses?

TweetTokenizer was strongest overall for the tested social-media structures; spaCy was strong for URLs and emojis but weak for hashtags and mentions; Word Tokenize generated the most tokens but was weakest at preserving social-media-specific structures.

12. Overall Conclusion

The experiment demonstrates that tokenizer choice has a substantial effect on processing informal social-media text. NLTK TweetTokenizer performed best overall for the features tested: it preserved hashtags, mentions, and URLs completely and achieved high emoji preservation. spaCy showed strong URL and emoji performance but did not preserve hashtags and mentions as complete tokens. NLTK Word Tokenize generated the largest number of tokens, but its high token count partly resulted from splitting social-media-specific structures. Therefore, the results indicate that a higher token count should not automatically be interpreted as better tokenization. For this dataset and evaluation criterion, TweetTokenizer is the most suitable of the three tested approaches.

13. Limitations

- The experiment uses 50 posts, so results should not be generalized to all social-media text.
- The source dataset is Tesla-focused and may not represent other social-media domains.
- Emoji tokenization is affected by Unicode representation, especially for compound emojis.
- The slang/abbreviation category is not exhaustively represented by a fixed list of expressions.
- The evaluation uses a strict exact-token preservation criterion; alternative definitions of correctness could produce different measurements.

14. Project Outputs

- `data/task1_posts.csv` — selected experimental posts.

- `data/task1_ground_truth.csv` — extracted ground-truth social-media elements.
- `results/tokenizer_results.csv` — structured tokenizer outputs.
- `results/tokenizer_output.txt` — detailed per-post tokenization output.
- `results/evaluation_summary.csv` — quantitative evaluation results.
- `results/error_analysis.txt` — identified tokenization errors.

15. AI Usage Statement

AI assistance was used for coding assistance, methodology guidance, regular-expression design, tokenizer implementation, debugging, and explanation of NLP concepts. AI was also used to help identify potential public datasets and structure the experimental workflow. The final dataset selection, program execution, tokenizer outputs, quantitative measurements, error analysis, and conclusions were generated and verified by running the implemented code. Experimental numerical results were not fabricated or assumed by the AI. AI suggestions were critically evaluated during development, including adjustments made when candidate data sources or implementation steps proved impractical. AI therefore served as a coding and learning aid, while the reported findings are based on the actual experimental data and execution.